Megan Pokal
Nasheb Ismaily
DSC650-T301 Big Data (2263-1)
February 1, 2026

Objective 1 – Conceptual Foundations:

HBase Architecture

HBase

For HBase, the HMaster manages the cluster by assigning regions to region servers, balancing the load, and monitoring overall health. Region servers are the worker nodes that handle client read and write requests, with each server managing one or more regions. Which are ranges of rows from a table. Data is then stored as HFiles in HDFS, allowing HBase to be scalable and fault-tolerant, since regions can be reassigned if a server fails. ZooKeeper coordinates the system by tracking the active region server and then communicating directly with it. HBase is a wide-column store that organizes data into tables, rows, and column families, where related columns are grouped and stored together on disk. HBase is good for real-time workload.

HBase Row Key Design

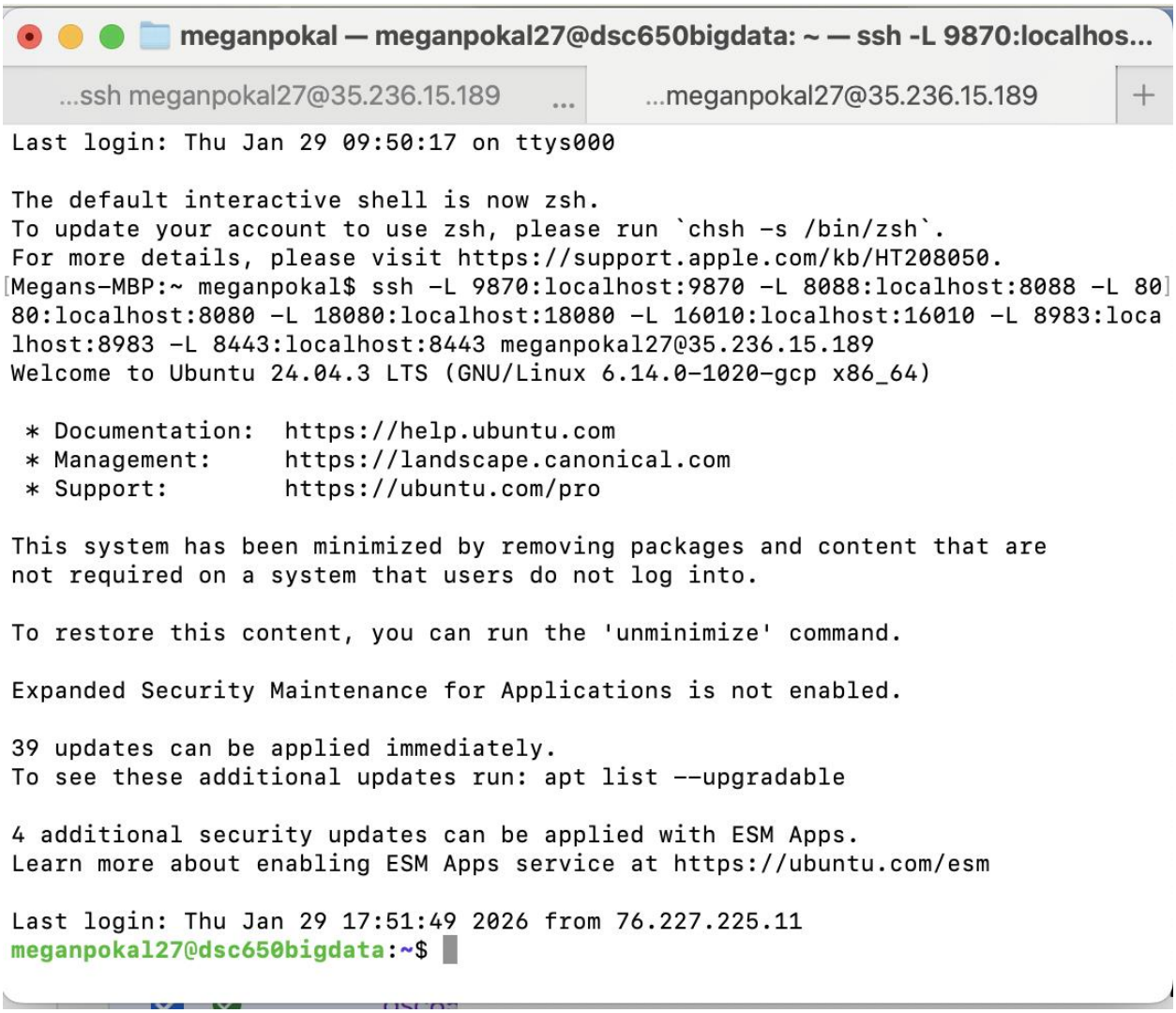
In HBase, data is stored and read based on the row key, so a poorly designed row key can cause serious performance problems. Using predictable keys like timestamps or sequential numbers creates hotspots where one server handles most of the work, while others sit idle. To avoid this, row keys should include a random prefix or salt to spread data evenly across regions. Additionally, columns that are frequently accessed together should be placed in the same column family, since each column family is stored separately on disk.

SQL vs NoSQL

Traditional SQL databases like MYSQL, PostgreSQL, and Oracle are relational databases. They store data in tables with fixed schemas and support transactions, making it ideal for structured data and workloads that require consistency. NoSQL databases like HBase's are designed for flexibility and horizontal scalability, as it can handle large amounts of data by spreading it across many servers. HBASE is a wide-column database that provides fast access to massive datasets without requiring a fixed schema. SQL and NoSQL databases solve different problems, like SQL is best for structured and relational data. While NoSQL focuses on scalability and flexibility with big data.

Objective 2 – HBase Table Creation and Management:

ssh -L 9870:localhost:9870 -L 8088:localhost:8088 -L 8080:localhost:8080 -L 18080:localhost:18080 -L 16010:localhost:16010 -L 8983:localhost:8983 -L 8443:localhost:8443 meganpokal27@35.236.15.189



1. Environment Initialization

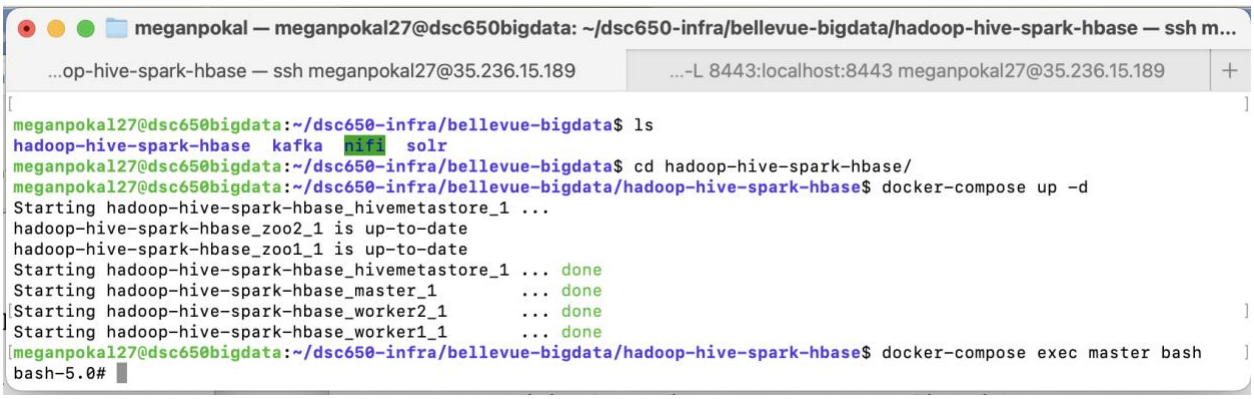
- Start by navigating to the required directory and initiating the Docker containers:

  cd dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase

  docker-compose up -d

- Access the master container:

  docker-compose exec master bash

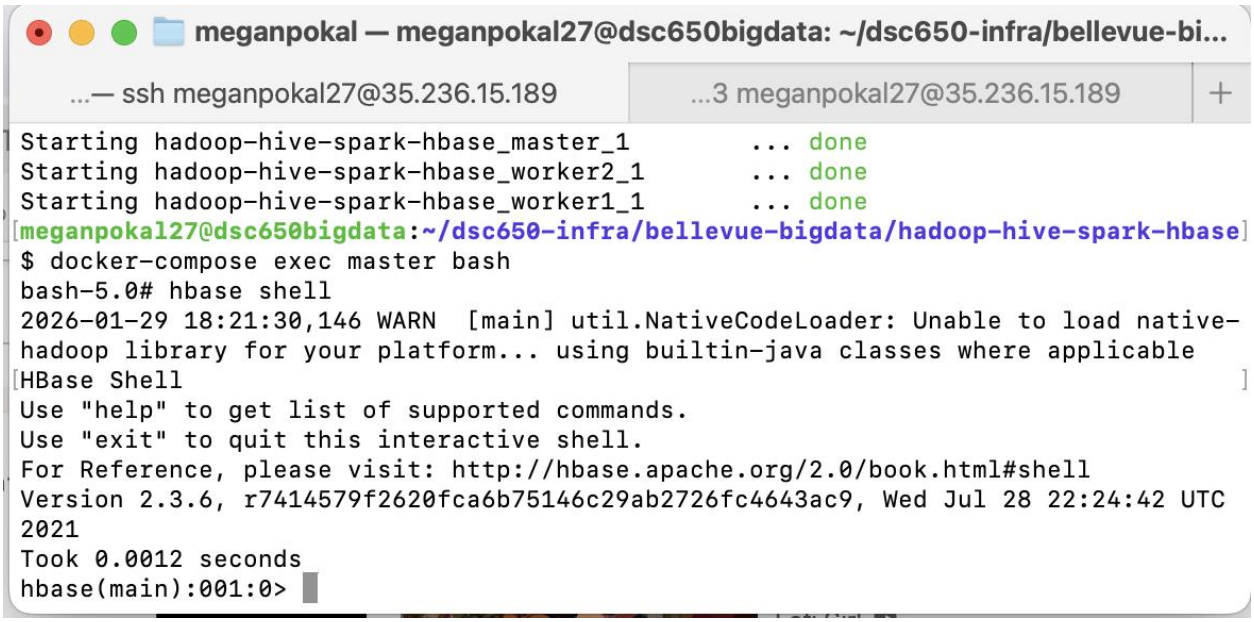


2. Introduction to HBase

Enter the HBase interactive shell:

hbase shell

Starting up the HBase shell



3. Table Creation and Management

Exercise 1: Create a table named ‘students’ with a column family ‘details’.

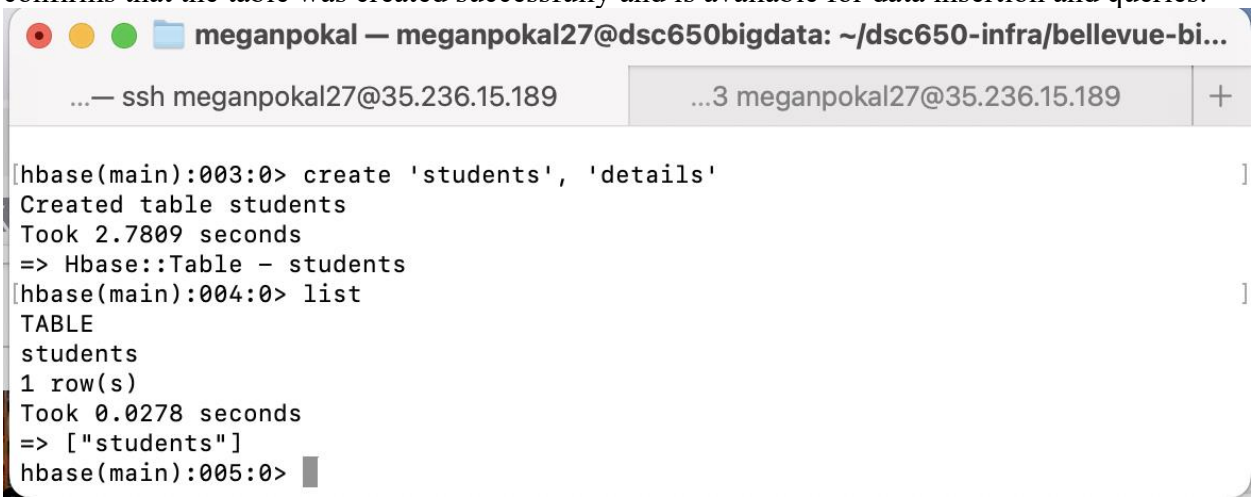
create 'students', 'details'

The output confirms that the students table was successfully created in HBase with a column family named details. The tables is now ready to store student information using row keys and column based data storage.

Exercise 2: Verify that the table has been created.

list

The list command displays all existing tables in HBase. Seeing the students table in the output confirms that the table was created successfully and is available for data insertion and queries.



Objective 3 – HBase Data Manipulation

1. Data Manipulation in HBase

Exercise 3: Add data to the 'students' table. Let's assume each student has a unique ID, a first name, and a last name.

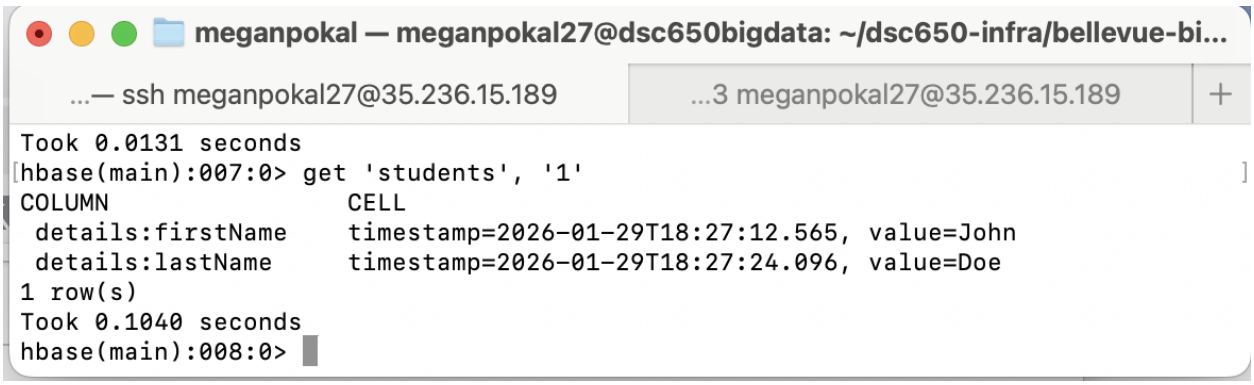
put 'students', '1', 'details:firstName', 'John'
put 'students', '1', 'details:lastName', 'Doe'

The students first and last names are stored as key-value pairs under the details column family. T his is how HBase stores data by row key and column.

Exercise 4: Query the data from the 'students' table to retrieve the details of the student with ID '1'.

get 'students', '1'

The output shows that the students first name and last name were successfully inserted into the st udents table and are stored under the details column family.



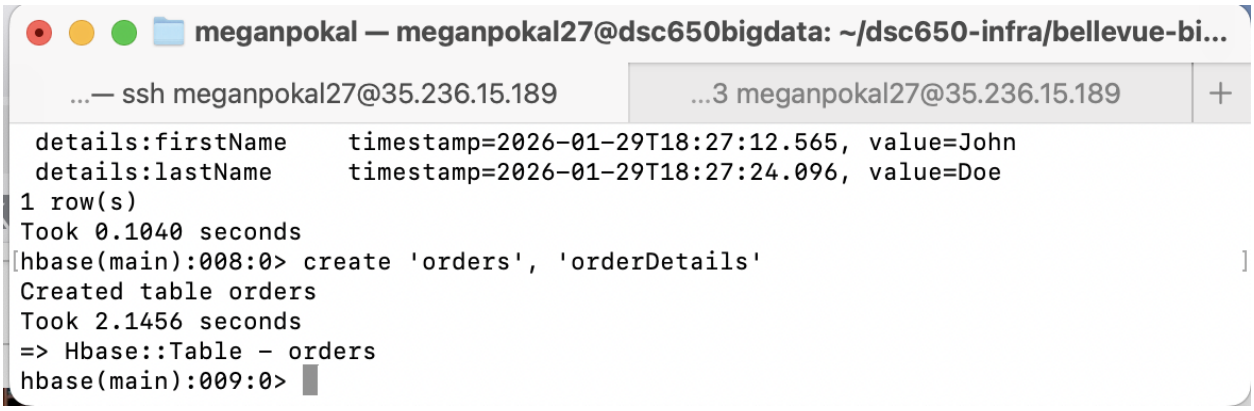
Objective 4 – Composite Row Keys with HBase

1.Advanced HBase Features: Composite Row Key

Exercise 5: Create a table named ‘orders’ to store data about customer orders. Assume each order is uniquely identified by a composite key formed by combining the customer ID and order date (in the format YYYYMMDD).

create 'orders', 'orderDetails'

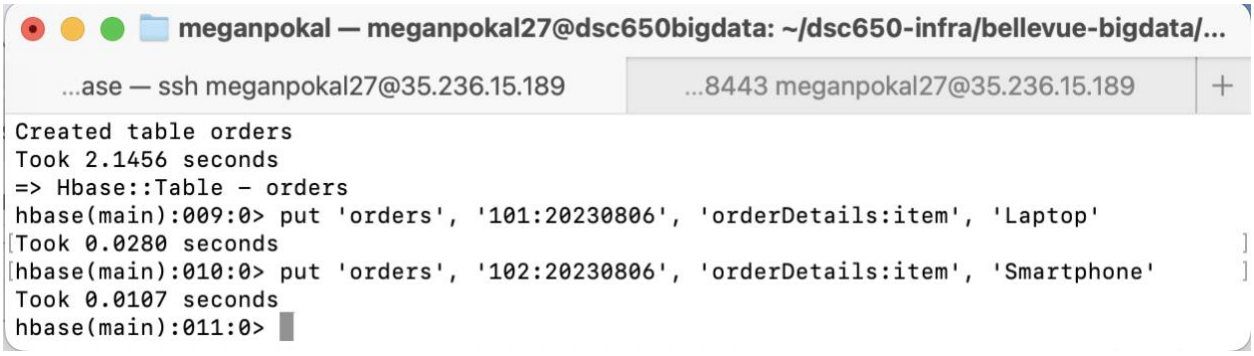
It combined the customer ID and order data, allowing related orders to be stored together. Also, r etrieving all orders for a specific customer.



Exercise 6: Add sample data to the ‘orders’ table using the composite key:

put 'orders', '101:20230806', 'orderDetails:item', 'Laptop'
put 'orders', '102:20230806', 'orderDetails:item', 'Smartphone'

Each row key is a composite key made up of the customer ID and the order date in YYYYMMDD format.

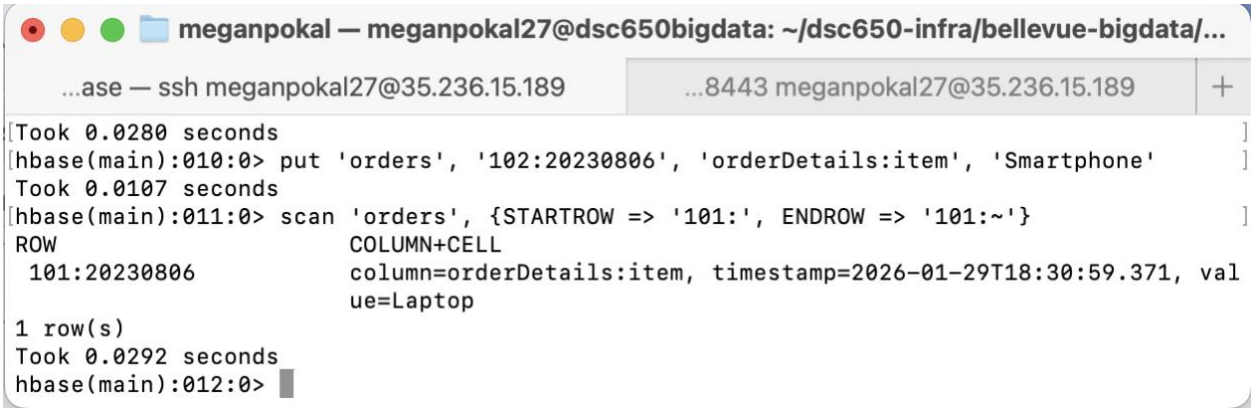


Exercise 7: Query the 'orders' table to retrieve details of all orders placed by the customer with ID '101'.

scan 'orders', {STARTROW => '101:', ENDROW => '101:~'}

This command will scan rows starting from '101:' to before '101:~' (tilde '~' is the next ASCII character after colon ':').

The scan has filtered and returned all orders associated with customer ID 101.



Objective 5 – Data Generation in HBase

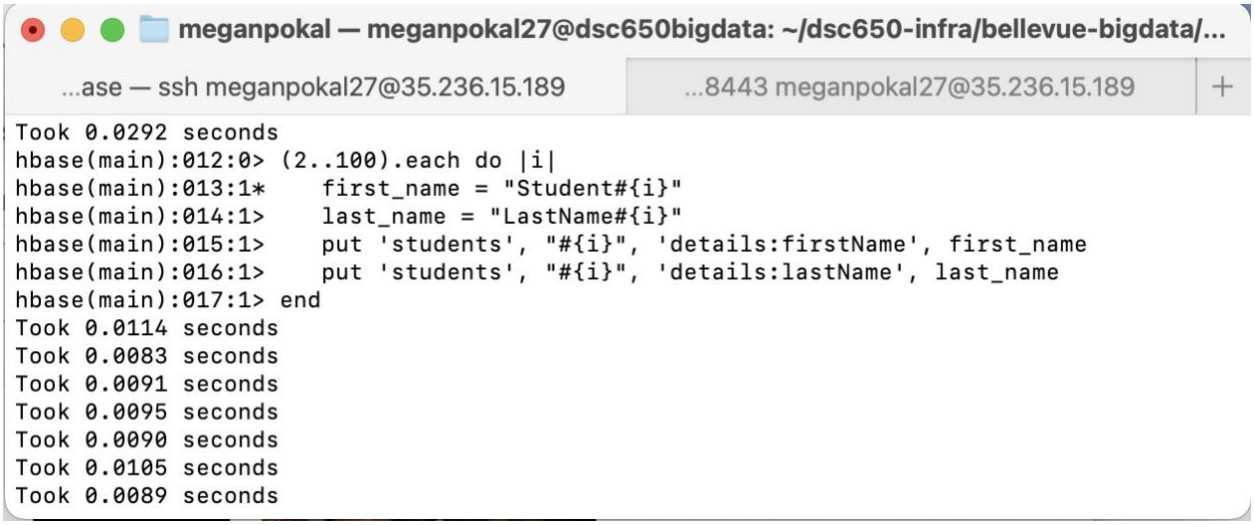
1. Data Generation for HBase

Exercise 8: Generate random data for the 'students' table.

```
(2..100).each do |i|
  first_name = "Student#{i}"
  last_name = "LastName#{i}"
```

```
  put 'students', "#{i}", 'details:firstName', first_name
  put 'students', "#{i}", 'details:lastName', last_name
end
```

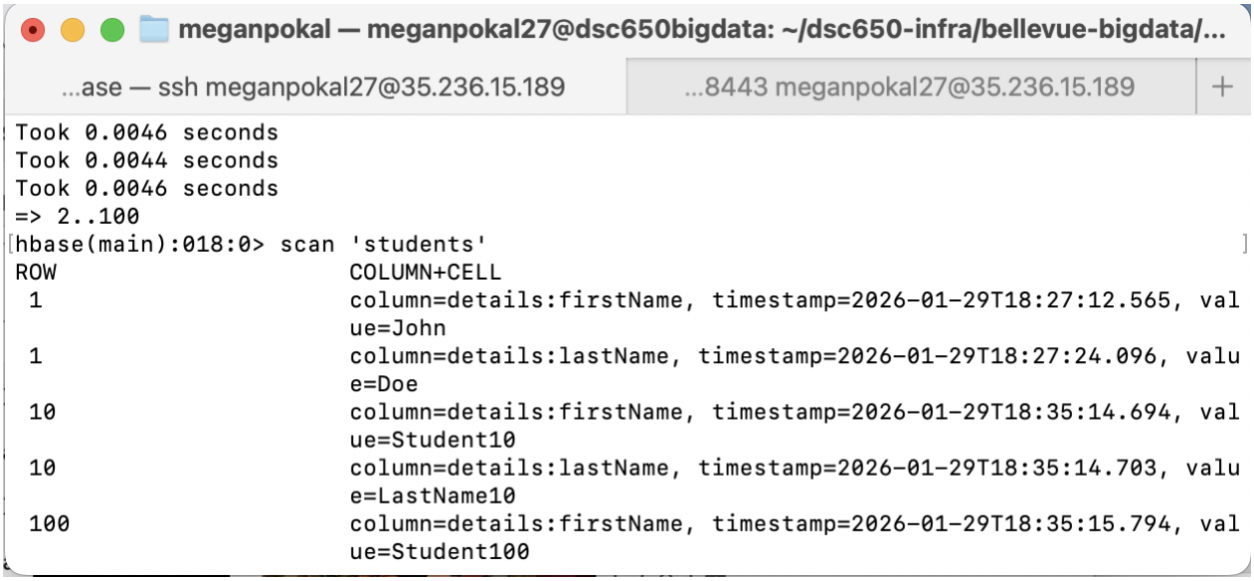
This creates a loop through student IDs 2 through 100 and generates sample data for each rown in the students table. Inserting it into the details:firstName and details:lastName columns.

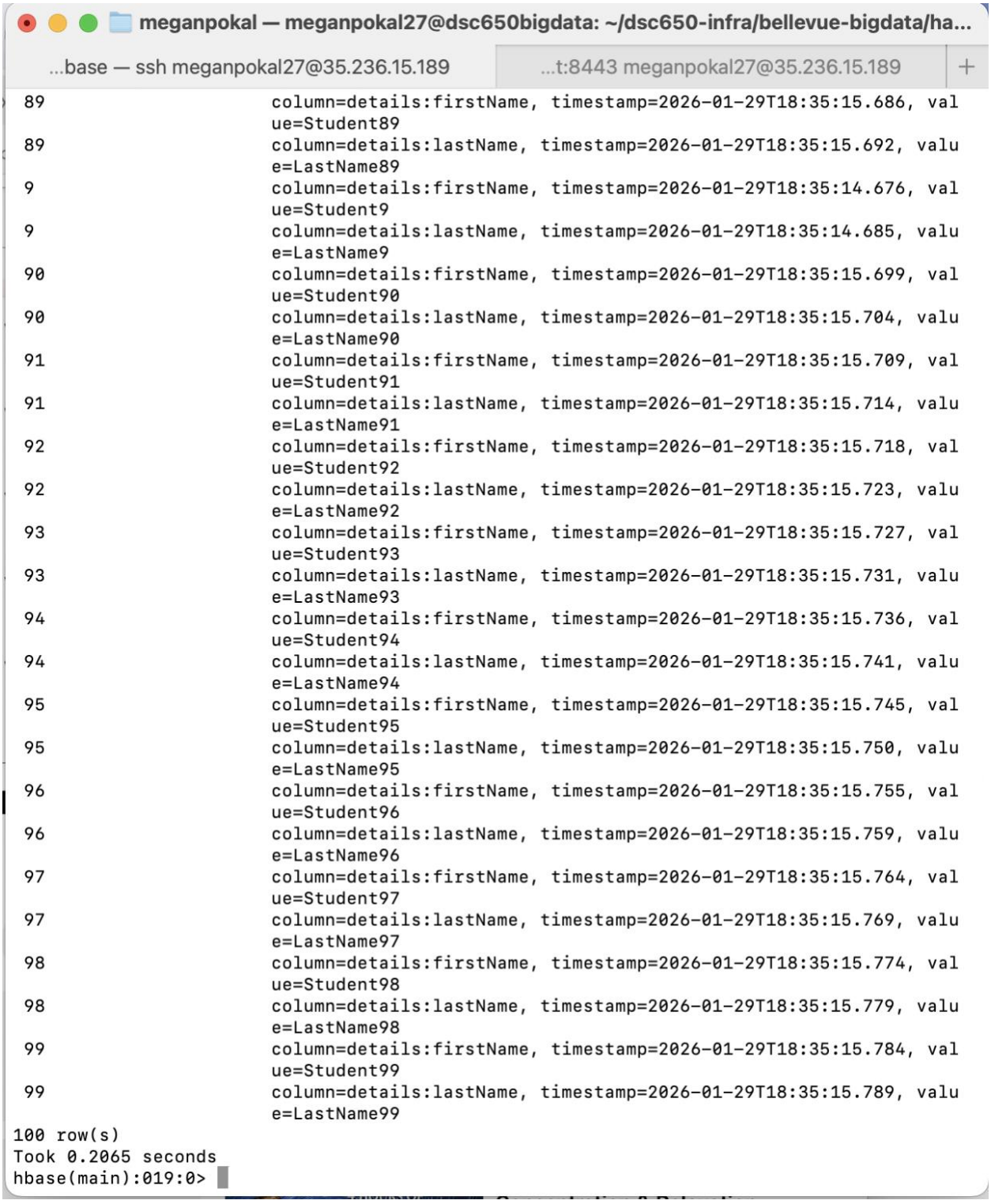


Exercise 9: Scan the ‘students’ table to verify data insertion.

```
scan 'students'
```

This makes sure that the students table was successful and each row represents a unique student I D and displays the correct columns under details, like firstname, lastname and middle name.


meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/...
...ase — ssh meganpokal27@35.236.15.189
...8443 meganpokal27@35.236.15.189
Took 0.0046 seconds
Took 0.0044 seconds
Took 0.0046 seconds
=> 2..100
hbase(main):018:0> scan 'students'
ROW COLUMN+CELL
1 column=details:firstName, timestamp=2026-01-29T18:27:12.565, value=John
1 column=details:lastName, timestamp=2026-01-29T18:27:24.096, value=Doe
10 column=details:firstName, timestamp=2026-01-29T18:35:14.694, value=Student10
10 column=details:lastName, timestamp=2026-01-29T18:35:14.703, value=LastName10
100 column=details:firstName, timestamp=2026-01-29T18:35:15.794, value=Student100



Objective 6 – Advanced Data Manipulation in HBase

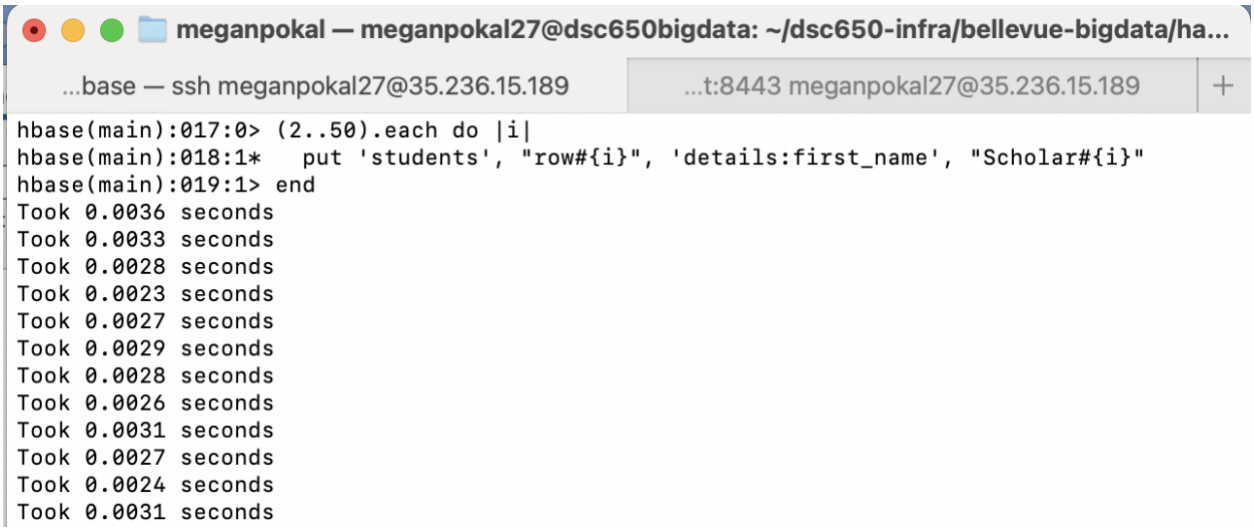
Exercise 10: HBase Data Manipulation

1. **Update First Names:**

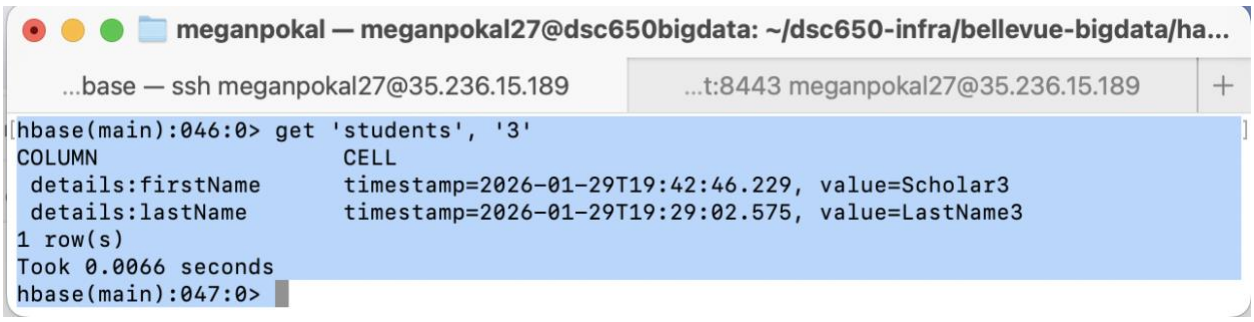
– For students with IDs from 2 to 50, change the first name prefix from Student to Scholar. For instance, Student3 should become Scholar3.

```
(2..50).each do |i|
  put 'students', "row#{i}", 'details:first_name', "Scholar#{i}"
end
```

This update replaces the existing first name values for students with IDs 2 - 50 by overwriting the column using the put command. HBase updates data by writing a new version of the cell




```
get 'students', '3'
```



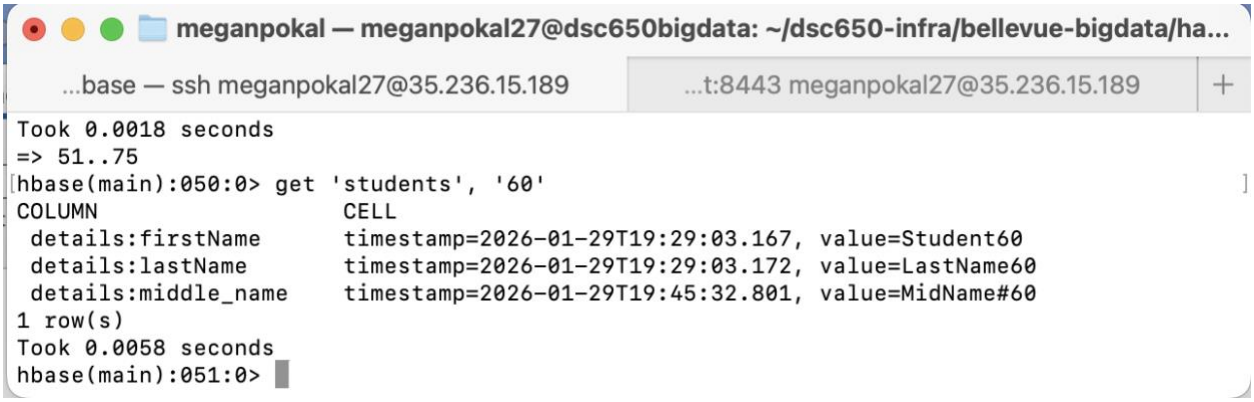
Add a Middle Name:

– For students with IDs from 51 to 75, add a middle name column under the details column family. The middle name should follow the pattern MidName#{i}.

```
(51..75).each do |i|
  put 'students', "row#{i}", 'details:middle_name', "MidName##{i}"
end
```

This will add a new column, middle_name, under the details column family for students with IDs 51-75. HBase will allow us to create a column without altering the table schema.

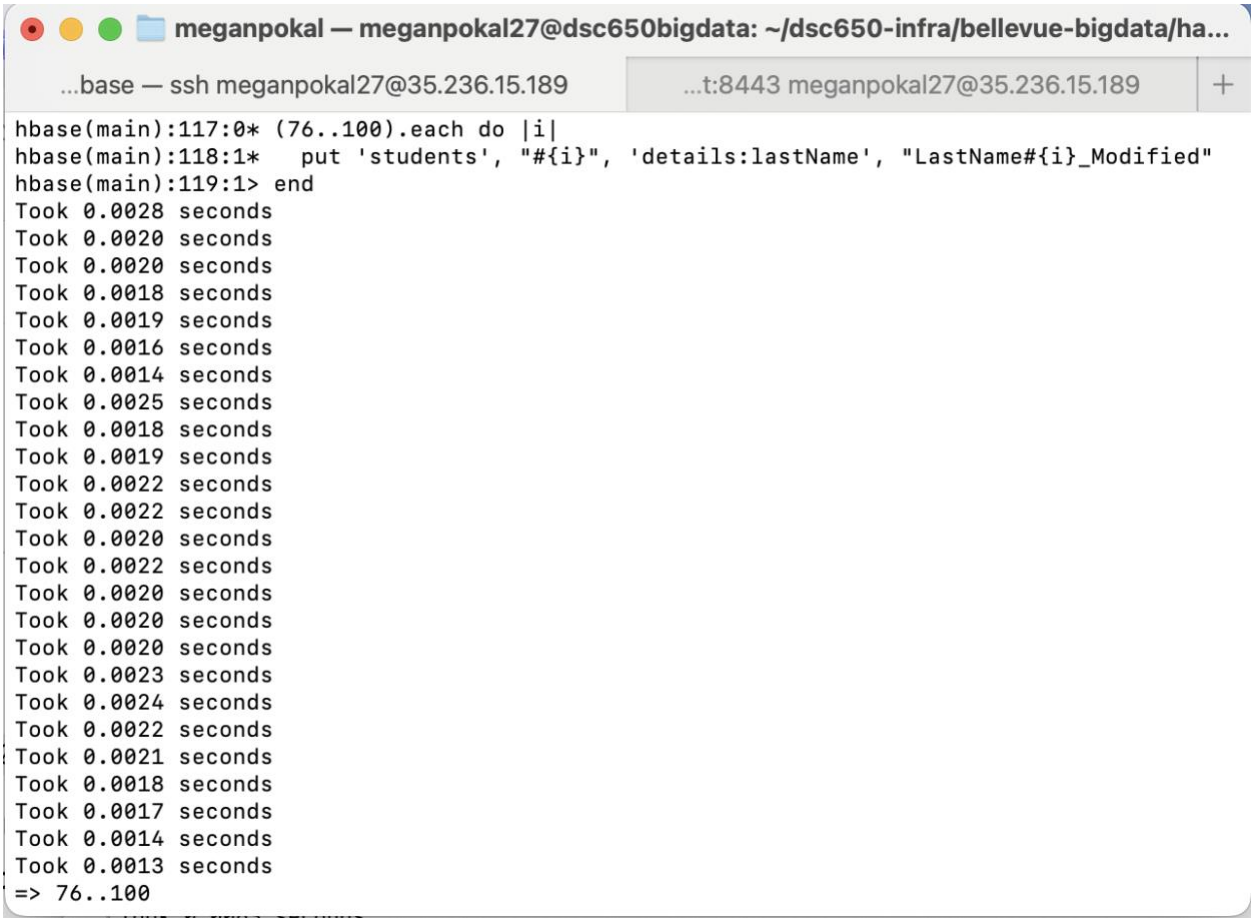
get 'students', '60'



Modify Last Names:

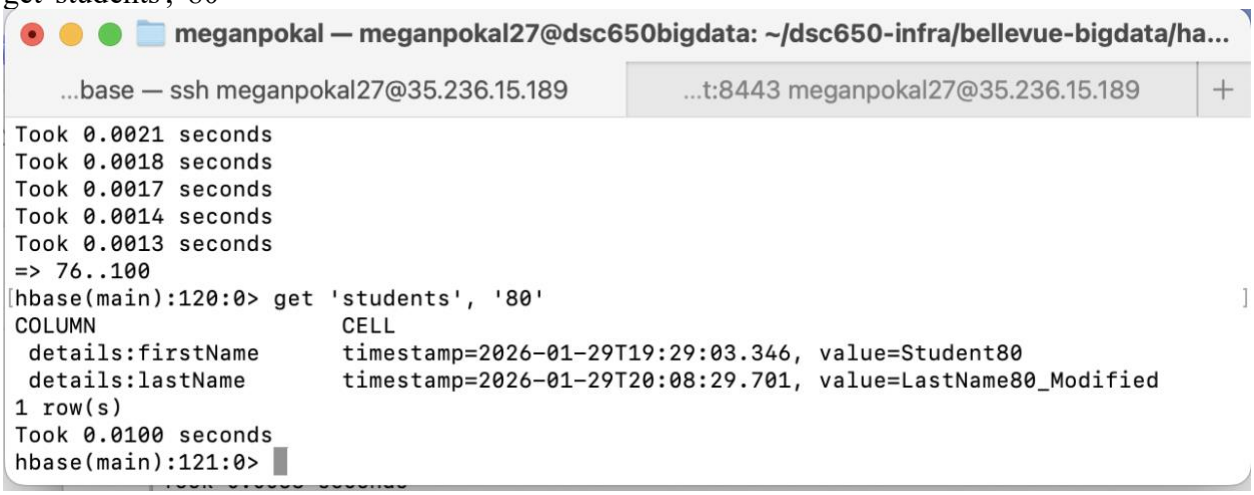
- For students with IDs from 76 to 100, append _Modified to the last name. So, LastName76 should be updated to LastName76_Modified.

```
(76..100).each do |i|
  put 'students', "#{i}", 'details:last_name', "LastName#{i}_Modified"
end
```



The last name for each student from ID 76 to 100 was updated by appending _Modified. This was done by overwriting the existing last_name column values using put.

get 'students', '80'

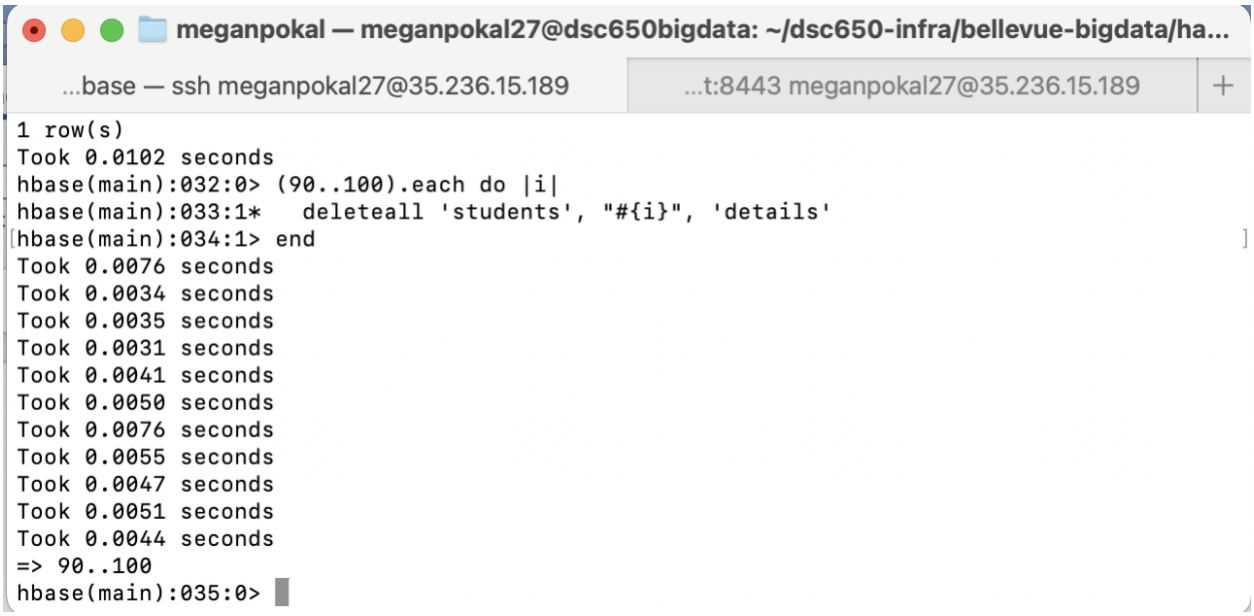


Bulk Delete:

- Delete all the details for students with IDs from 90 to 100.

```
(90..100).each do |i|
  deleteall 'students', "#{i}", 'details'
end
```

The bulk delete removed all columns under the details column family for students with IDs 90-100. The deleteall command permanently removes the specified column family data from each row.



get 'students', '90'

Run get, row 90 to make sure that it has been successfully deleted.

Data Retrieval:

- After all modifications, retrieve and display the details for students with IDs 40, 60, 80, and 90 to verify changes.

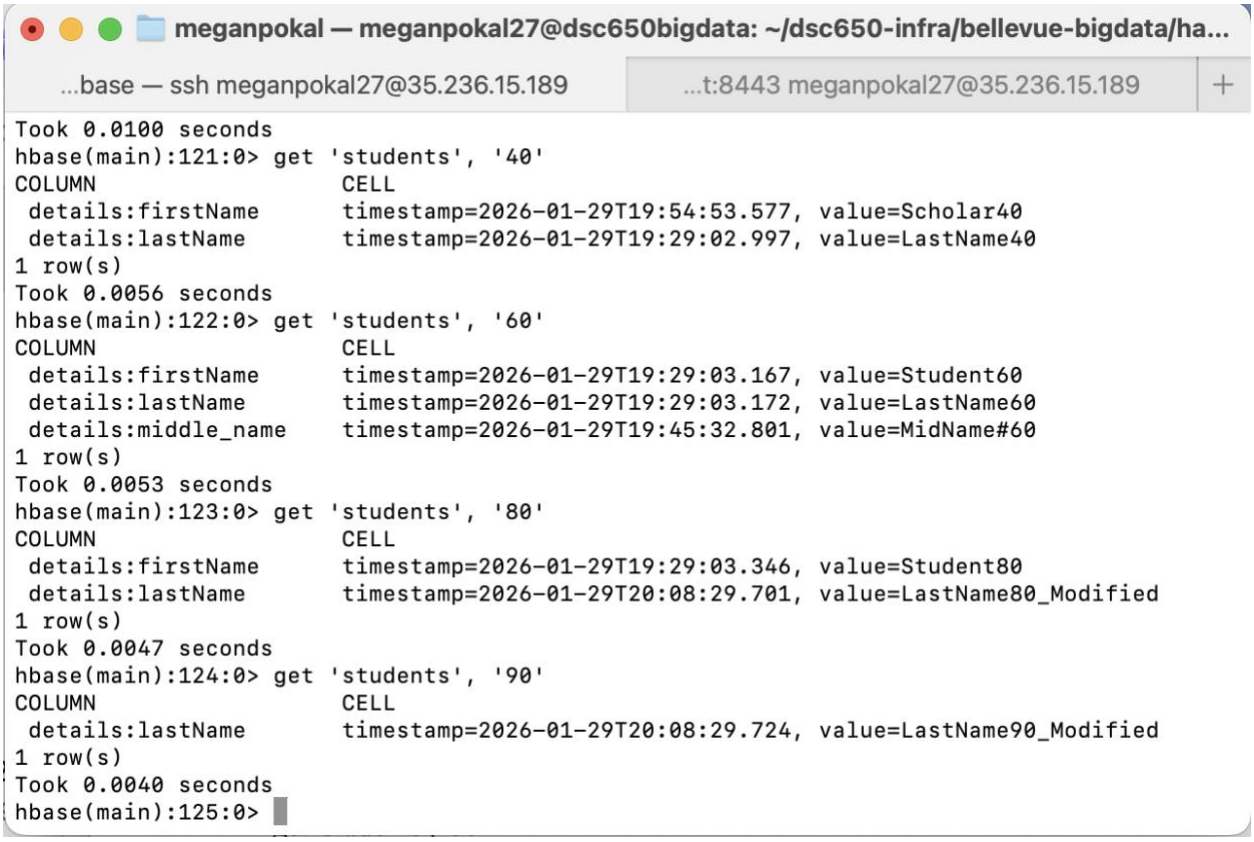
get 'students', '40'

get 'students', '60'

get 'students', '80'

get 'students', '90'

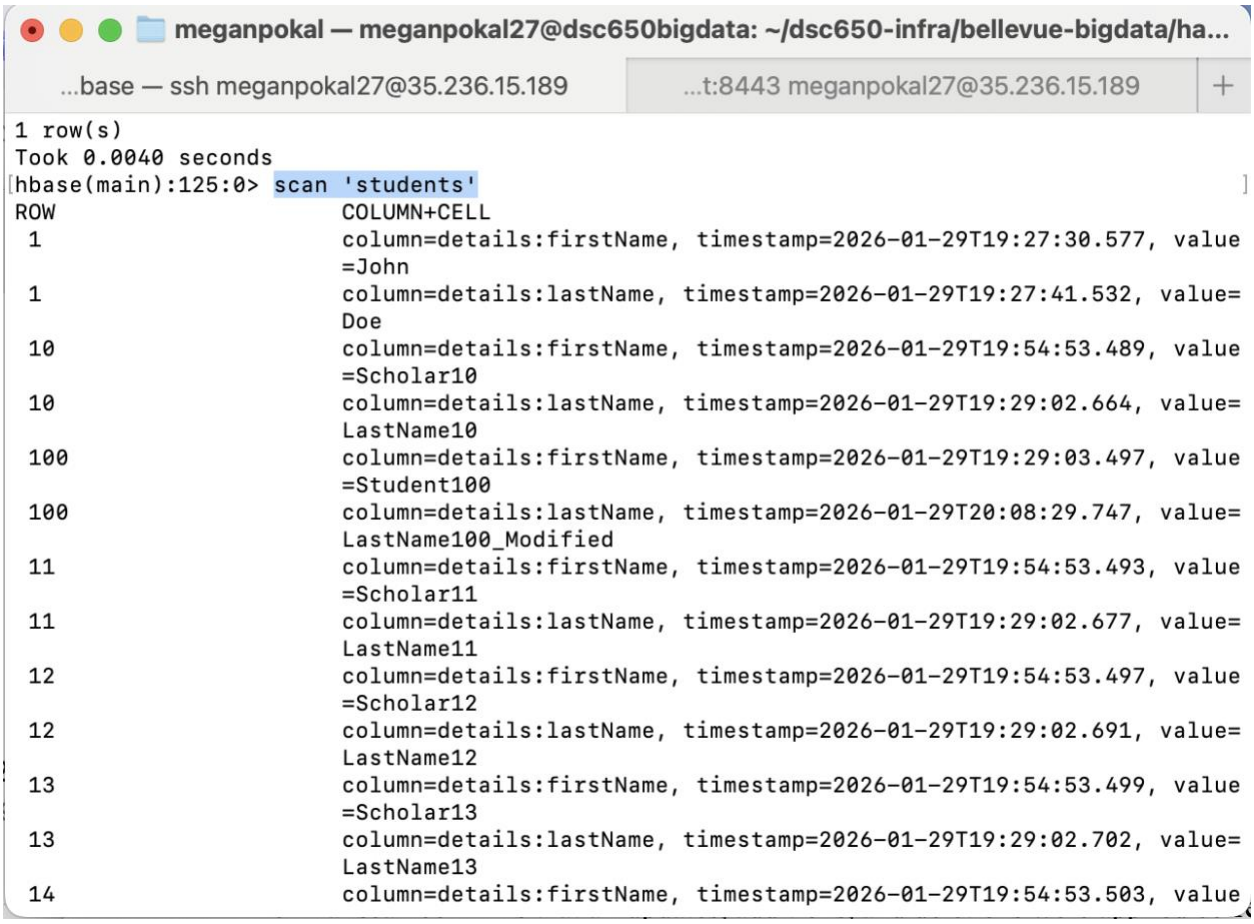
This will show that all the different commands have been completed correctly. Showing firtName updated, firstName unchanged, middle_name, lastName modified, and then the last part has been deleted and there are no rows.

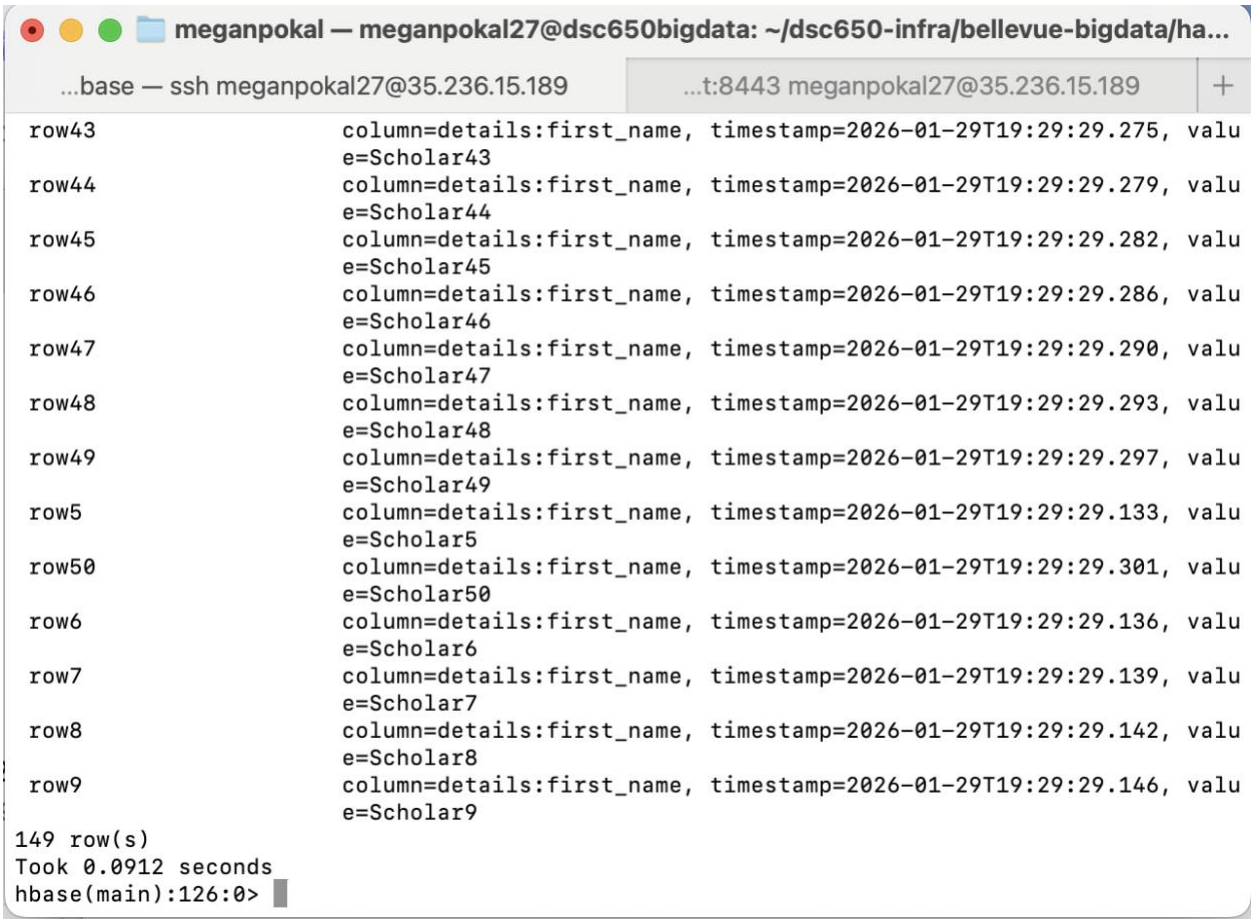


Final Table Scan

scan 'students'

The final scan confirms that all updates, additions, and deletions were applied correctly. The table reflects all the changes that I have implemented.


meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/ha...
...base — ssh meganpokal27@35.236.15.189
...t:8443 meganpokal27@35.236.15.189
1 row(s)
Took 0.0040 seconds
hbase(main):125:0> scan 'students'
ROW COLUMN+CELL
1 column=details:firstName, timestamp=2026-01-29T19:27:30.577, value=John
1 column=details:lastName, timestamp=2026-01-29T19:27:41.532, value=Doe
10 column=details:firstName, timestamp=2026-01-29T19:54:53.489, value=Scholar10
10 column=details:lastName, timestamp=2026-01-29T19:29:02.664, value=LastName10
100 column=details:firstName, timestamp=2026-01-29T19:29:03.497, value=Student100
100 column=details:lastName, timestamp=2026-01-29T20:08:29.747, value=LastName100_Modified
11 column=details:firstName, timestamp=2026-01-29T19:54:53.493, value=Scholar11
11 column=details:lastName, timestamp=2026-01-29T19:29:02.677, value=LastName11
12 column=details:firstName, timestamp=2026-01-29T19:54:53.497, value=Scholar12
12 column=details:lastName, timestamp=2026-01-29T19:29:02.691, value=LastName12
13 column=details:firstName, timestamp=2026-01-29T19:54:53.499, value=Scholar13
13 column=details:lastName, timestamp=2026-01-29T19:29:02.702, value=LastName13
14 column=details:firstName, timestamp=2026-01-29T19:54:53.503, value


meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/ha...
...base — ssh meganpokal27@35.236.15.189
...t:8443 meganpokal27@35.236.15.189
row43 column=details:first_name, timestamp=2026-01-29T19:29:29.275, value=Scholar43
row44 column=details:first_name, timestamp=2026-01-29T19:29:29.279, value=Scholar44
row45 column=details:first_name, timestamp=2026-01-29T19:29:29.282, value=Scholar45
row46 column=details:first_name, timestamp=2026-01-29T19:29:29.286, value=Scholar46
row47 column=details:first_name, timestamp=2026-01-29T19:29:29.290, value=Scholar47
row48 column=details:first_name, timestamp=2026-01-29T19:29:29.293, value=Scholar48
row49 column=details:first_name, timestamp=2026-01-29T19:29:29.297, value=Scholar49
row5 column=details:first_name, timestamp=2026-01-29T19:29:29.133, value=Scholar5
row50 column=details:first_name, timestamp=2026-01-29T19:29:29.301, value=Scholar50
row6 column=details:first_name, timestamp=2026-01-29T19:29:29.136, value=Scholar6
row7 column=details:first_name, timestamp=2026-01-29T19:29:29.139, value=Scholar7
row8 column=details:first_name, timestamp=2026-01-29T19:29:29.142, value=Scholar8
row9 column=details:first_name, timestamp=2026-01-29T19:29:29.146, value=Scholar9
149 row(s)
Took 0.0912 seconds
hbase(main):126:0>